

smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 18 日

目录

1	smallgameagent 实验报告 (终稿)	2
1.1	0. 一页结论	2
1.2	1. 环境打通 (新信息)	3
1.3	2. 空间一致性 (问题 (1))	3
1.4	3. 策略优化 (问题 (3))	4
1.5	4. 效率 (问题 (4))	5
1.6	5. 时间一致性 (问题 (2))	5
1.7	6. 本地 VLM 画像 (P5)	5
1.7.1	6.1 Intel 核显 Vulkan (旧基线)	5
1.7.2	6.2 RTX 5060 Laptop CUDA + KV-cache 量化 (新基线)	5
1.7.3	6.3 结论	6
1.8	7. Agent 通信与记忆 (P8)	6
1.8.1	实验状态	6
1.9	8. 云端 API 实验状态 (P8)	7
1.10	9. 训练准备 (P7, 等 ssh5090)	7
1.11	10. verifiers 风格环境 (P6)	7
1.12	11. 给同学四个问题的直接回答	7
1.13	12. 后续建议 (优先级序)	8

1 smallgameagent 实验报告 (终稿)

> 2026-07-18。配套：__CODE__0000__ (方案)、__CODE__0001__ (过程数据)。
> 本文面向合作同学与老师，按四个问题组织结论，附 RTX 5060 本地模型、Agent 通信/记忆、云端 API 实验状态。

1.1 0. 一页结论

- __BOLD__0000__：根因不是「记不住场景」而是 __BOLD__0001__。实证：
00461 场景切换后 __CODE__0000__ 激活阻断摇杆输入，规则 agent 继续发移动指令
瘫痪 175+ 步。
解法是 __BOLD__0000__ (stale 标记、失败面板自动点击)，
A/B 显示活动率 0.54→0.92、墙钟 263→154s (−42%)。

- __BOLD__0000__：交付 __BOLD__0001__ (三层时间对齐 + settle 复查 + 受
保护前缀 hash +
失败分级 rollback/compensation/stop+replan)，29 单测含真实温度案例复刻 (131/190
步偏差)。

- __BOLD__0000__：kimi-k2.7-code 的经济决策实验显示 __BOLD__0001__
(4K max_tokens 下 58% 调用截断无答案)；修正后朴素与引导式在饱和场景打平 (0.93)，
决定性场景 (batch 占优 3-4×) 结果见 §3。

- __BOLD__0000__：动态探针预算模块 (五触发器、L0/L1/L2 分层、L2 预算上
限 20%) + 动态日志分级，
27 单测，模拟装饰性场景节省观测成本 ~82%。

- __BOLD__0000__：
 • Intel 核显 Vulkan：4B 2.75 tok/s，视觉 struct 4 帧解析率 0/4，仅适合作离线标
注。
 • __BOLD__0000__：4B 文本 __BOLD__0001__、视觉 struct 8 帧解析率 __BOLD__0002__、
target_acc __BOLD__0000__、平均 __BOLD__0001__；gemma-4-E4B 文本 __BOLD__0002__。
8 GB 显存可跑 4B，
E4B 接近上限 (7082 MB)。

- __BOLD_0002__: 新增显式消息总线 __CODE_0000__、循环流水线 __CODE_0001__、文件型 __CODE_0000__、__CODE_0001__ 角色与 __CODE_0002__ / __CODE_0003__ 模式；19 单测覆盖。

- __BOLD_0001__: OpenCodeGo 认证通过但余额不足 (__CODE_0000__) , mimo-v2.5 / kimi-k2.7-code /

kimi-k2.6 矩阵待充值后跑；当前在线 gameplay 亦受 WSL2 headless Chromium 无法初始化 Cocos 场景阻塞。

- __BOLD_0000__: processed-runs 已转为 7 任务 15,083 样本（14 单测 + 加载验证），

训练脚本就绪，等 ssh5090 可访问后开 QLoRA。

1.2 1. 环境打通（新信息）

项	结论
云端 LLM	__CODE_0000__, kimi-k2.7-code 文本决策 2.4–6.3s/次（旧 deepseek 8–15s），多模态可用；
本地 VLM	LM Studio 捆绑 llama.cpp Vulkan 后端（WSL2 Mesa Dozen→Intel Graphics）；lms CLI 因版本
游戏基线	rule 模式 SSD_00461 塔防 300 步基线 composite 0.425（verifiers 风格 rubric）

1.3 2. 空间一致性（问题 (1)）

__BOLD_0000__（三级递进，全部有 run 数据）：

- 初判「场景切换后撞新障碍卡死」→ 障碍学习 + 势场避让（P1）上线后仍失败；
- 再判「英雄死亡」→ 诊断跑血量全程 100，排除；
- __BOLD_0000__。

这正是同学说的「交互方式改变」类失效（摇杆交互被模态面板替换）。

__BOLD_0000__：

- __CODE_0000__(__CODE_0001__):entity_version/scene_epoch/capability_epoch,

观测写入比对、stale 标记、局部重规划，12 单测（含 autoFishing 12 次翻转复刻）。

- 探针新增 __CODE_0000__: 按 __BOLD_0001__ 匹配按钮（生产构建组件名被 minify 成单字母，

按类名匹配必然落空), 返回设计分辨率坐标; Python 侧 retry 优选、避开 download 广告陷阱,

设计坐标 → CSS 像素映射 (720×1560 左下原点 → 375×812 顶左)。7 单测。

- RuleEngine 障碍学习 (势场避让 + 定向逃逸, 15 单测) ——对几何卡死有效, 但非本游戏 binding。

__BOLD_0000__:

run	composite	activity	progress	stall 步
基线	0.425	0.535	0.667	139
+ 世界模型/障碍	0.351	0.408	0.583	177
__BOLD_0000__	__BOLD_0000__	__BOLD_0000__	__BOLD_0000__	__BOLD_0000__

1.4 3. 策略优化 (问题 (3))

实验: 12 经济场景 (gold/双升级项/收入/预算), 最优解由穷举模拟器给出, kimi-k2.7-code

朴素 prompt vs 机制引导 prompt (先抽象经济循环 + 往返成本再决策)。

- v1 (max_tokens 4096): __BOLD_0000__——思考型模型把 token 预算耗尽于推理,

JSON 被截断。教训: 思考模型必须留足输出预算 (16K) 并约束「末行只放 JSON」。

- v2 (16K tokens): 解析失败归零; 朴素 vs 引导 0.929 vs 0.932 (场景饱和, 11/12 greedy 最优, 无法区分)。

- v3 (8 个 batch 占优 3-4× + 4 对照, 模拟器穷举真值):

__BOLD_0000__; 按得分比率 (所选策略得分/最优得分):

* 朴素 batch=0.711 / control=1.000 (overall 0.808); 引导 batch=1.000 / control=0.958 (overall 0.986) **。

结论: 机制引导把 batch 经济决策从 71% 提升到 100% 最优值, 代价仅 greedy 场景—4%——

定量证实了「加以引导后探索最优解能力强」的判断。引导的关键是 prompt 里显式给出

「经济循环 + 往返/机会成本」的抽象, 而不是让模型自己发现。

1.5 4. 效率（问题（4））

- __CODE_0000__: L0 状态 (73ms) /L1 组件 (150ms) /L2 截图 (400ms) 三级, 五触发器(phase_change/low_confidence/action_no_effect/collision_hint/semantic_flip), 冷却防抖 + L2 窗口占比上限（默认 20%）+ 高优先挤占; AdaptiveLogger 环形内存 DEBUG + 触发窗口落盘。27 单测; 50 步无变化场景模拟节省 81.75%。

- 失败面板自动恢复即「策略级回退」的首个实例（行为级：障碍逃逸；状态级：stale 重规划；策略级：面板 dismiss）。

1.6 5. 时间一致性（问题（2））

__CODE_0000__ (29 单测): TimestampedValue 三层时间戳(event/observed/settled)、跨字段一致性校验器（复刻 190 步 131 违例）、PhaseGate 状态机（settle 复查防完成假阳性、守卫违反 VIOLATED、超时 TIMEOUT）、受保护前缀 hash（篡改即 PrefixViolation）、失败按副作用分级。与游戏 loop 的集成（契约守卫替换硬编码 fail 检测）是下一步。

1.7 6. 本地 VLM 画像（P5）

1.7.1 6.1 Intel 核显 Vulkan（旧基线）

模型	server tok/s	wall 60 tokens	备注
Qwen3.5-4B	2.75	24.6 s	可用
Qwen3.5-9B	0.72	90.3 s	慢但可跑
gemma-4-E4B-it	0.67	37.5 s	输出格式偏闲聊，需强约束

- 4 帧视觉 struct: 解析率 0/4, 平均 534 s/帧（超时/连接抖动）。

1.7.2 6.2 RTX 5060 Laptop CUDA + KV-cache 量化（新基线）

后端: __CODE_0000__
参数: __CODE_0000__

Qwen3.5 为思考模型, 必须给足 __CODE_0000__ (2048) 才能拿到 content JSON; 否则 256/900 tokens

模型	文本 tok/s	60 tokens wall	显存占用	视觉 struct
Qwen3.5-4B-Q4_K_M	___BOLD_0000___	1.0 s	3465 MB	8 帧解析率 ___BOLD_0000___
gemma-4-E4B-it-Q4_K_M	___BOLD_0000___	0.7 s	7082 MB	单帧 14.6 s, 输出带 fence 但

会被 reasoning 占满。

1.7.3 6.3 结论

- CUDA 后端让 4B 文本速度提升 ___BOLD_0000___、视觉延迟从 10+ min 降到 ___BOLD_0001___, 8 GB 显存跑 4B 宽裕,

E4B 已接近上限。

- 在线控制仍建议使用云端 mimo-v2.5 (~9.5 s/帧) 或更大显存; 本地 4B 适合离线标注与

低成本原型验证。

1.8 7. Agent 通信与记忆 (P8)

新增机制:

模块	作用
___CODE_0000___	显式消息总线; ___CODE_0000___ 含 OBSERVE/PERCEIVE/DECIDE/VERIFY/C
___CODE_0000___	Observer → StateMapper → DecisionAnalyst → Verifier → Critic 循环, Verifier 可触
___CODE_0000___	文件型策略记忆, 按 ___CODE_0000___ 索引, 记录成功/失败次数
___CODE_0000___	Critic 角色, 给出 ___CODE_0000___ + ___CODE_0001___
___CODE_0000___	注册 ___CODE_0000___ / ___CODE_0001___ 模式

验证: 19 单测 (bus 5 + orchestrator 2 + strategy memory 4 + 既有决策 maker 集成)。

1.8.1 实验状态

- 在线 gameplay 矩阵 (rule / multi / multi-bus / multi-bus-memory / api / api-memory) 已编写

___CODE_0000___, 但当前 WSL2 headless Chromium 无法初始化 Cocos

场景 (___CODE_0000___ 为 null), 所有模式 steps=0。待 headed/GPU 环境或浏览器参数

修复后复跑。

1.9 8. 云端 API 实验状态 (P8)

- OpenCodeGo 客户端认证通过(____CODE_0000____),但调用返回 ____CODE_0001____。
- 因此 ____BOLD_0000____、____BOLD_0001____、____BOLD_0002____ 的混合实验当前无法实际跑通;已记录阻塞,

充值后可直接用 ____CODE_0000____ 扩展 API 配置。

1.10 9. 训练准备 (P7, 等 ssh5090)

- ____CODE_0000____: 22 游戏 3054 步 → ____BOLD_0001____

(next_probe_action 2645 / probe_action_effect 2645 / field_grounding 2645 / information_gain 3054 / pulse_response 1435 / progression 2645 / failure_recovery 14), VLMColdStartDataset 加载验证通过, 14 单测。

- failure_recovery 仅 14 条 (成功 run 中卡死窗口少) ——建议用 verifiers 环境跑对抗性失败 run 补数据 (rule 变体/随机扰动注入)。

- ____CODE_0000____ (QLoRA 4bit NF4 + ZeRO-2) 就绪; gemma4-e4b 需 transformers 补丁复核。
- ssh5090 可访问后: ____CODE_0000____ + 数据同步 + 开训。

1.11 10. verifiers 风格环境 (P6)

____CODE_0000____: score_trajectory 五轴 rubric (completion/progress/activity/consistency/composite), 可直接评分历史 run JSON, 也可 GameEnv.rollout 在线跑分。8 单测。为后续 RL 微调提供 env+rubric 抽象。

1.12 11. 给同学四个问题的直接回答

1. ____BOLD_0000____: 版本化世界模型 + 失效检测 (stale/面板/能力翻转) + 分级恢复;
关键是「交互方式失效」要有一等公民的检测通道 (本游戏是 failCount 翻转, 其他游戏可能是 autoFishing 类能力标志)。

2. ____BOLD_0000____: 三层时间戳对齐 + 阶段契约 (precondition/allowed_actions/success+settle/

timeout) + 前缀 hash 防策略改写 + 失败分级。

3. __BOLD_0000__: 引导式 prompt 的方向对, 但先把 __BOLD_0001__ (token 预算、末行 JSON、

解析回退) 做扎实; 最优性评估用模拟器穷举做真值, 别让模型自己评。

4. __BOLD_0000__: 默认 L0 状态探针 + 触发器升级; DEBUG 环形内存、触发窗口落盘;

回退分级 (行为/状态/策略)。

1.13 12. 后续建议 (优先级序)

1. 面板处理泛化: 把 LosePanel 个案抽象为「模态面板阻断」检测器 (任意 Panel 激活 + 输入无效化),

进阶阶段契约守卫。

2. ssh5090 QLoRA: 先 qwen3.5-4b (管线已验证), 数据用 cold-start + processed-runs 合并。

3. 用 verifiers 环境批量产 failure_recovery 数据 (对抗性失败注入)。

4. 9B/E4B 基准补齐; VL struct 准确率上去后替代 mimo 视觉降本。

5. 22 游戏 benchmark: 每游戏 profile + 3 seed $\times$ 2 配置 (机制开/关) 配对复跑。